ACTIVIDAD AUTÓNOMA

Django I

Proyecto, app y primer modelo

Trabajo individual guiado. Sigue las diapositivas en orden: cada paso termina con un checkpoint.



ANTES DE EMPEZAR

¿Qué es Django?

Un framework web de Python, de código abierto, con las **baterías incluidas**.

2003 Nace en la redacción del Lawrence Journal-World, un periódico de Kansas.

2005 Se libera como código abierto.

Hoy Lo mantiene la Django Software Foundation y una comunidad global.

Su nombre viene del guitarrista de jazz **Django Reinhardt**.

LO QUE TRAE DE FÁBRICA

ORM
consultas en Python, sin SQL a mano

Admin
panel de datos listo para usar

Auth
usuarios, sesiones y permisos

Migraciones
el esquema de la BD versionado

Seguridad
protección contra CSRF, XSS e inyección SQL

URLs
todas las rutas en un solo lugar

ANTES DE EMPEZAR

Quién lo usa en producción

Instagram

Su backend se construyó sobre Django; es una de las instalaciones de Django más grandes del mundo.

Mozilla

Sitios como el de complementos de Firefox, addons.mozilla.org, están hechos con Django.

Pinterest

Usó Django como base en sus primeros años de crecimiento.

Disqus

Su sistema de comentarios, incrustado en miles de sitios, se construyó con Django.

Lo que comparten: muchos datos, mucho tráfico y equipos que necesitaban avanzar rápido sin reinventar la base. Eso es lo que Django resuelve.

ANTES DE EMPEZAR

Hoy avanzas por tu cuenta, en seis pasos



100 min
el bloque de la clase del jueves

3
capturas, una por checkpoint, más tres preguntas

Martes
arrancamos desde tu proyecto ya migrado

CONTEXTO

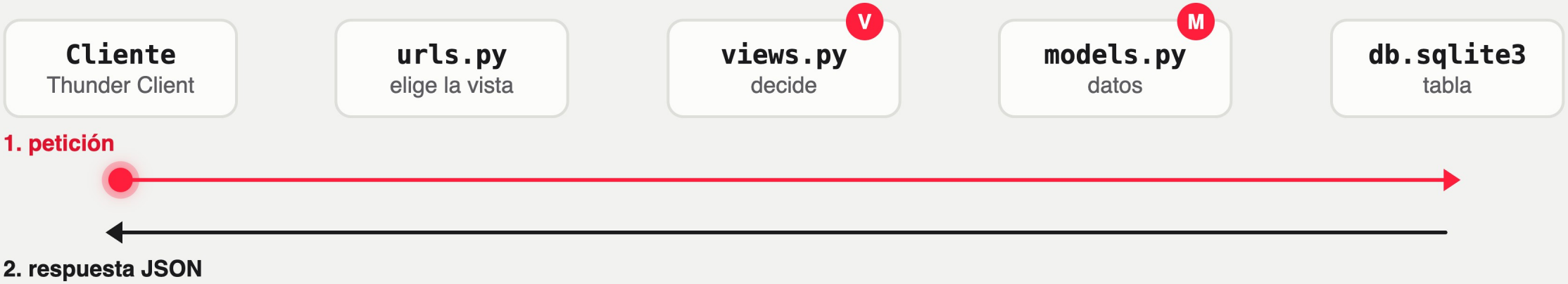
De Flask a Django: lo que antes armabas a mano

EN FLASK LO HICISTE TÚ	EN DJANGO YA VIENE
routes.py con un Blueprint	**urls.py + views.py**
servicio.py leyendo un JSON	**Modelo + ORM**
id calculado con max(id) + 1	**id que asigna la base de datos**
Catálogo en memoria que se pierde	**db.sqlite3, persistente en disco**
Sin panel para revisar datos	**Panel de administración (martes)**

Django decide la estructura por ti. A cambio, te toca aprender **dónde va cada cosa**.

CONCEPTO

MVT: el recorrido de una petición en Django



M Model

Datos y reglas. **Hoy solo tocamos esta.**

V View

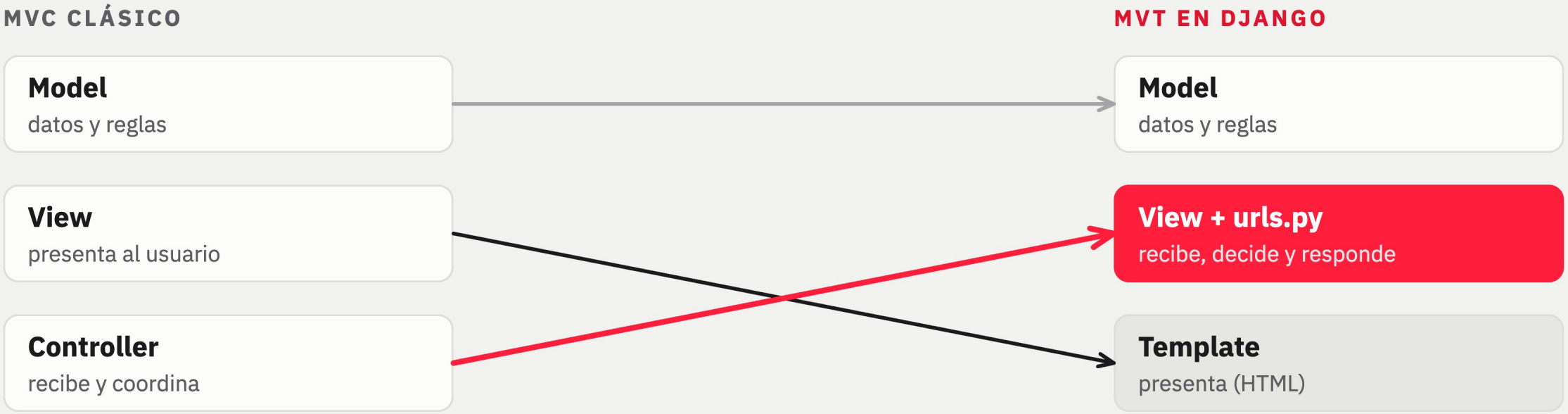
Recibe la petición, consulta y responde.

T Template

Arma HTML. Con DRF respondemos JSON.

CONCEPTO

¿Por qué MVT y no MVC?



Mismas responsabilidades, otros nombres. El **Controller** de MVC, en Django lo reparten la **View** y el framework, que con **urls.py** elige la vista. La View de MVC aquí se llama **Template**. Por eso Django habla de **MVT**.

CONCEPTO

Proyecto y app: dos piezas con trabajos distintos

catalogo-django/

`manage.py` `requirements.txt` `.gitignore`

PROYECTO
config/

Configuración global: base de datos, apps instaladas, rutas principales.

- `settings.py`
- `urls.py`

APP
catalogo/

Todo lo del catálogo de canciones vive aquí.

- `models.py`
- `views.py · admin.py`
- `migrations/`

MÁS APPS, DESPUÉS

- `usuarios/`
- `playlists/`
- `reseñas/`

Un proyecto crece sumando apps, cada una con un solo propósito.

PASO 1 DE 6

Carpeta nueva, entorno nuevo

terminal

```
$ mkdir catalogo-django
$ cd catalogo-django
$ python -m venv .venv
# Windows (PowerShell)
$ .venv\Scripts\Activate.ps1
# macOS / Linux
$ source .venv/bin/activate
$ pip install django
$ pip freeze > requirements.txt
$ python -m django --version
```

.gitignore

```
.venv/
```

Proyecto separado

No reutilices el entorno de Flask: cada proyecto lleva sus propias dependencias.

CHECKPOINT La terminal muestra (.venv) y el último comando imprime la versión de Django.

PASO 2 DE 6

Crear el proyecto y arrancarlo

terminal

```
$ django-admin startproject config .
```

El **punto final** crea el proyecto en la carpeta actual.

✓ **con punto**

```
catalogo-django/
├── config/
└── manage.py
```

⚠ **sin punto**

```
catalogo-django/
└── config/
    └── config/
```

127.0.0.1:8000



The install worked successfully!

Captura esta página

CHECKPOINT Captura 1: la página de bienvenida. Detén el servidor con Ctrl + C.

PASO 3 DE 6

Crear la app y registrarla

terminal

```
$ python manage.py startapp catalogo
```

config/settings.py

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    # ... las demás que ya vienen
    "catalogo",  # ← agrega esta línea
]
```

LO QUE DJANGO VE

admin	auth
contenttypes	sessions
messages	staticfiles

✓ **catalogo**

⚠ Sin registrar: **No changes detected** en el paso 5.

CHECKPOINT Existe la carpeta catalogo/ y aparece en INSTALLED_APPS.

PASO 4 DE 6

El modelo Cancion, en catalogo/models.py

catalogo/models.py

```python
from django.db import models
class Cancion(models.Model):
    spotify_id = models.CharField(max_length=22, unique=True)
    titulo = models.CharField(max_length=300)
    artista = models.CharField(max_length=200)
    album = models.CharField(max_length=300, blank=True)
    genero = models.CharField(max_length=50)
    popularidad = models.PositiveSmallIntegerField(default=0)
    duracion_ms = models.PositiveIntegerField()
    # El CSV mezcla "2019-06-14" con "2012"
    fecha_lanzamiento = models.CharField(max_length=10, blank=True)
    creada_en = models.DateTimeField(auto_now_add=True)
    class Meta:
        ordering = ["-popularidad"]
        verbose_name_plural = "canciones"
    def __str__(self):
        return f"{self.titulo} ({self.artista})"
```

id automático

No lo declaras: Django lo agrega y la base de datos lo asigna.

Atributo = columna

El tipo de campo decide el tipo en SQL y sus validaciones.

Mismo hilo

titulo y artista vienen de Flask; el CSV lo mapeamos el martes.

PASO 4 DE 6

Cada campo es una decisión de diseño

`max_length=22`

Los identificadores de Spotify miden 22 caracteres. El límite documenta el dato y lo valida.

`unique=True`

Impide dos filas con el mismo spotify_id. Lo vas a cuestionar en la pregunta 3.

`blank=True`

Permite dejar vacío el campo en formularios y en el admin. Sirve para álbum y fecha.

`PositiveSmallIntegerField`

La popularidad va de 0 a 100: un entero pequeño y sin negativos basta.

`auto_now_add=True`

Guarda la fecha de creación de la fila una sola vez, sin que tú la envíes.

`Meta y __str__`

Orden por defecto en las consultas y cómo se ve cada canción en el shell y el admin.

PASO 5 DE 6

Migraciones: del modelo a la tabla



La migración se versiona en Git; la base de datos no.

terminal

```
$ python manage.py makemigrations catalogo
$ python manage.py sqlmigrate catalogo 0001
$ python manage.py migrate
```

sqlmigrate

Solo muestra el SQL que se va a ejecutar; no cambia nada.

CHECKPOINT Captura 2: la salida de migrate, todas las líneas en OK.

PUENTE CON SQLITE3

El SQL que no escribiste

TU MODELO (PYTHON)	LO QUE EJECUTA SQLITE
`(no lo declaraste)`	`"id" integer PRIMARY KEY AUTOINCREMENT`
`CharField(max_length=22, unique=True)`	`"spotify_id" varchar(22) NOT NULL UNIQUE`
`CharField(max_length=300)`	`"titulo" varchar(300) NOT NULL`
`PositiveSmallIntegerField(default=0)`	`"popularidad" smallint unsigned CHECK (≥ 0)`
`DateTimeField(auto_now_add=True)`	`"creada_en" datetime NOT NULL`

Busca la fila del **id**: ahí está la respuesta a las dos preguntas que dejamos abiertas con max(id) + 1. La tabla se llama **catalogo_cancion**: app y modelo, en minúsculas.

PASO 6 DE 6

Probar el ORM desde el shell

python manage.py shell

```
>>> from catalogo.models import Cancion
>>> Cancion.objects.create(
...     spotify_id="6f807x0ima9a1j3VPbc7VN",
...     titulo="I Don't Care",
...     artista="Ed Sheeran",
...     genero="pop",
...     popularidad=66,
...     duracion_ms=194754,
... )
>>> Cancion.objects.count()
>>> Cancion.objects.filter(artista="Ed Sheeran")
>>> Cancion.objects.get(id=1).titulo
```

Crea **al menos tres canciones** con filas reales del CSV.

id	titulo	artista
1	I Don't Care	Ed Sheeran
2	[tu canción]	[artista]
3	[tu canción]	[otro artista]

Prueba repetir un spotify_id

```
IntegrityError: UNIQUE constraint failed:
catalogo_cancion.spotify_id
```

CHECKPOINT Captura 3: el shell con tus creaciones y consultas. Sal con exit().

SI ALGO FALLA

Errores comunes

`django-admin no se reconoce`

El entorno virtual no está activado. Revisa que la terminal muestre (.venv).

`No changes detected`

La app no está en INSTALLED_APPS, o no guardaste models.py.

`no such table: catalogo_cancion`

Corriste makemigrations pero te faltó migrate.

`That port is already in use`

Otro servidor sigue abierto. Ciérralo o usa runserver 8001.

`UNIQUE constraint failed`

Intentaste repetir un spotify_id. Es el comportamiento esperado.

`PowerShell no deja activar`

Ejecuta Set-ExecutionPolicy -Scope CurrentUser RemoteSigned y vuelve a activar.

Lee el mensaje de error completo antes de buscar ayuda: casi siempre dice qué falta.

PARA TU PDF

Tres preguntas para responder

1 En Flask el id salía de max(id) + 1. Con el SQL de sqlmigrate: ¿quién asigna ahora el id y por qué eso resuelve las dos preguntas abiertas, peticiones simultáneas y reinicio del servidor?

2 ¿Por qué db.sqlite3 y .venv/ van en el .gitignore, y en cambio la carpeta migrations/ sí se versiona?

3 Abre el CSV de 30,000 canciones y revisa si un mismo track_id aparece más de una vez. Con lo que encuentres, ¿unique=True en spotify_id es buena decisión para la carga del martes? Justifica.

Respuestas breves, con tus palabras: se valora el razonamiento, no la extensión.

ENTREGA EN MOODLE

Qué subes y cuándo

El proyecto en .zip

La carpeta catalogo-django **sin .venv/ ni db.sqlite3**, con requirements.txt y migrations/.

Un PDF

Las **tres capturas** de los checkpoints y las respuestas a las tres preguntas.

Declaración de IA

Al final del PDF: qué pediste a la IA y qué revisaste o cambiaste, o que **no la usaste**.

FECHA LÍMITE

Lunes 28 de septiembre, 23:59

Evidencia de clase: sin portada y sin calificación, pero obligatoria.